Groth16

Prepared by Kirill Sizov

Preliminary syllabus

- Groth16
- Circom
- Tornado Cash
- Railgun
- Halo2
- zkEVM

Agenda today

- General about ZK
- Groth16

zkSNARK

zero-**k**nowledge **S**uccinct **N**on-interactive **AR**guments of **K**nowledge

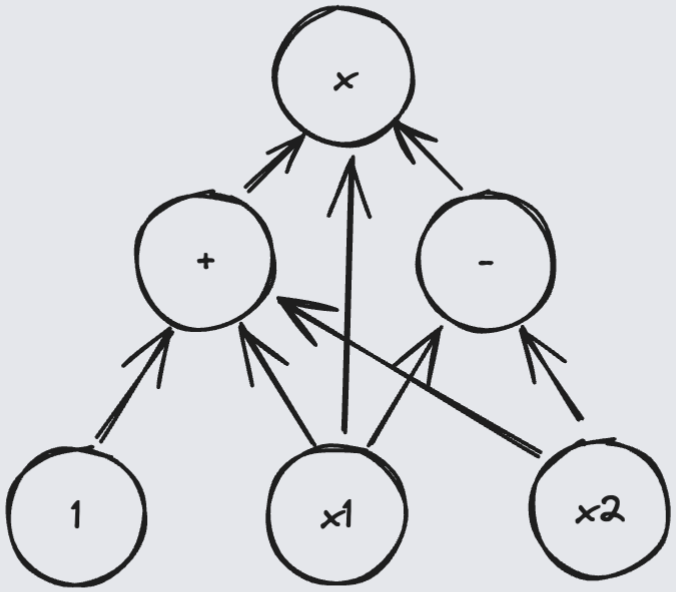
- zk: What the verifier can compute **after** interaction
  = What the verifier could compute **before** interaction
- Succinct: The proofs are small and easy to verify, regardless of the complexity of the statement.
- Non-Interactive: Does not require multiple rounds of communication between prover and verifier, making it efficient for decentralized systems.
- Argument of Knowledge: Proof that the prover knows the inputs from which the output was derived.

Circuits

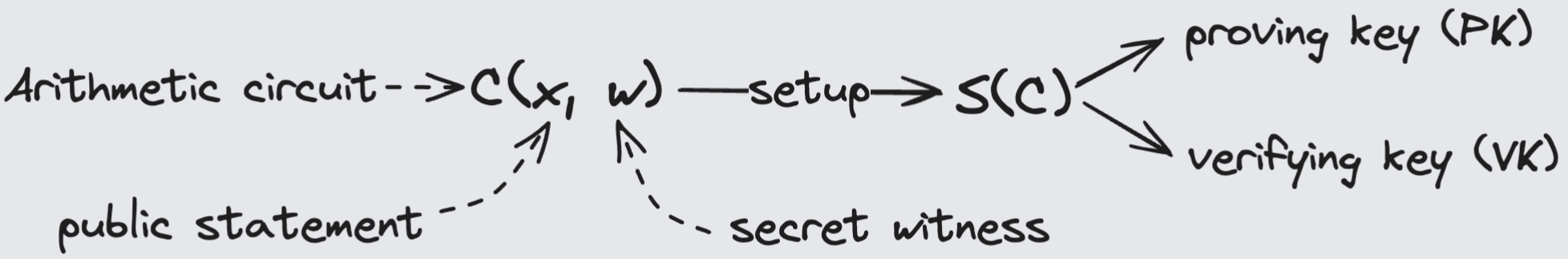
Arithmetic circuit

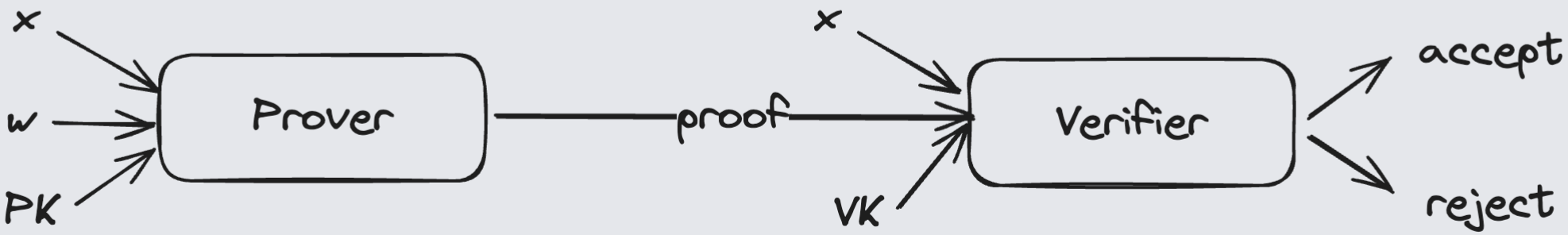
- Directed Acyclic Graph (DAG) with internal nodes representing operations (+, -, *) and leaves serving as input values.

$$(1 + x1 + x2) * x1 * (x1 - x2)$$



SNARK


Arithmetic circuit
C(x, w)
setup
S(C)
proving key (PK)
verifying key (VK)
public statement
secret witness


x
w
PK
Prover
proof
x
VK
Verifier
accept
reject

Setup types

- Trusted setup per circuit
  - Relies on a secret that should be kept from provers.
  - Example: Groth16.
- Universal trusted setup:
  - Can be reused across multiple circuits or applications.
  - Example: PLONK.
- Transparent setup
  - Don't use secret data.
  - Example: zk-STARK.

Groth16

Basic zero knowledge proof with elliptic curves

- Claim: I know two values x and y such that $x + y = C$
- Prover: Send $A = xG$ and $B = yG$ to verifier, where G is generator
- Verifier: Checks that $A + B = C \cdot G$

Bilinear pairing

- Bilinear pairing allows to prove that a 3rd number is the product of the original numbers without revealing them.
- Bilinear pairing is a map $e : G_1 \times G_2 \rightarrow G_T$ which satisfy bilinearity, non-degeneracy, efficiency.

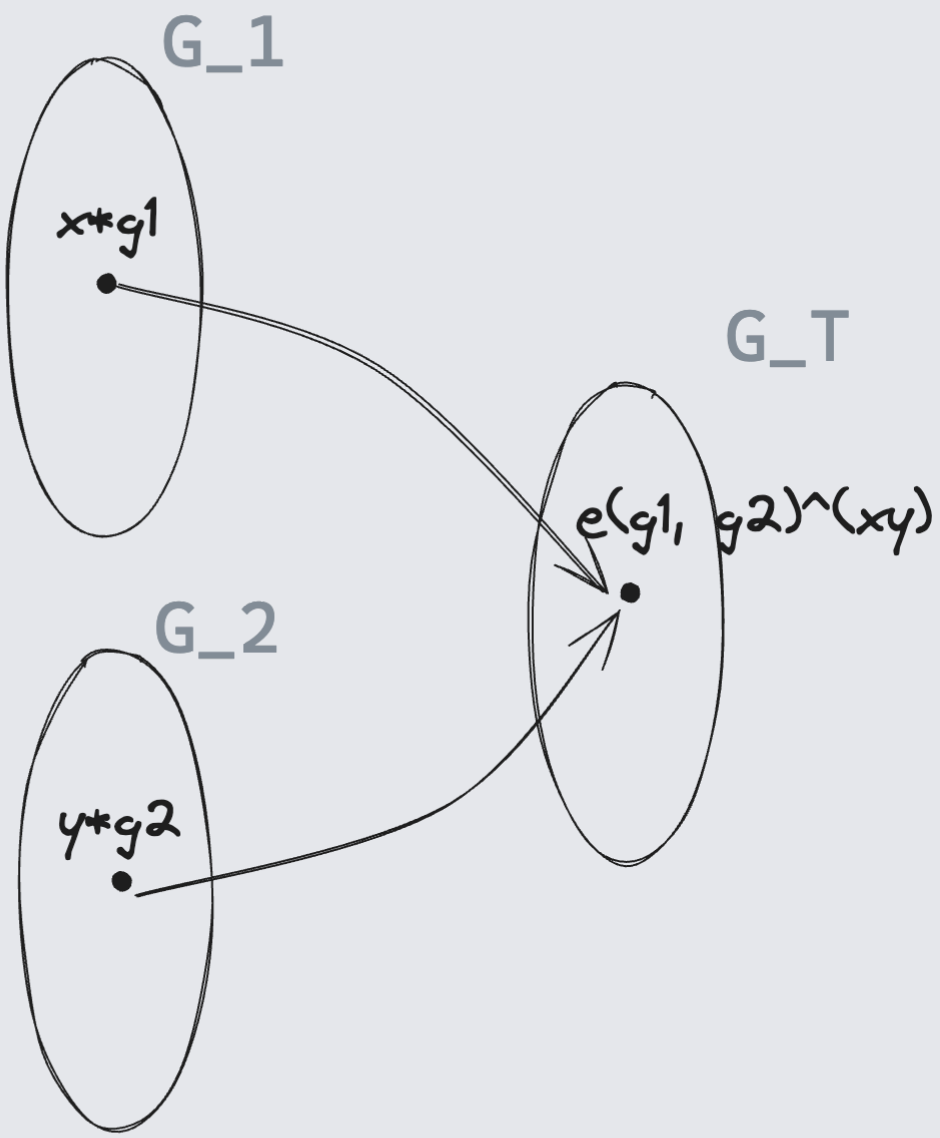
Bilinear pairing

- Bilinearity: $e(P+Q,S) = e(P,S) \cdot e(Q,S)$ and $e(P,S+T) = e(P,S) \cdot e(P,T)$
- Non-degeneracy: $e(P,Q) = 1_{G_T}$ only when $P = 0_{G_1}$ or $Q = 0_{G_2}$

From bilinearity, it follows that if g_1 is a generator of G_1, and g_2 is a generator of G_2, then $e(Ag_1, Bg_2) = e(g_1, g_2)^{AB}$.

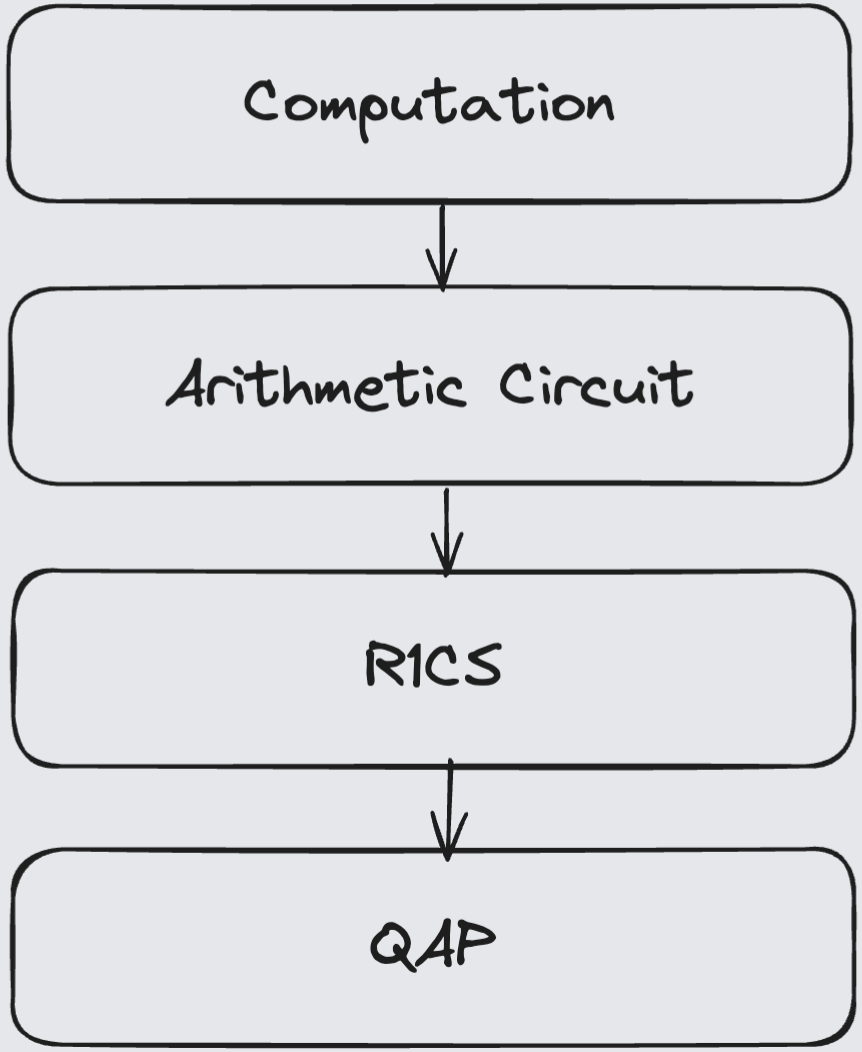
Bilinear pairing

- Claim: I know two values x and y such that $xy = D$
- Prover: Send $A = xg_1$ and $B = yg_2$ to verifier
- Verifier: Checks that $e(A, B) = e(g_1, g_2)^D$



* g1 and g2 are generators in G_1 and G_2

General steps



Flattening

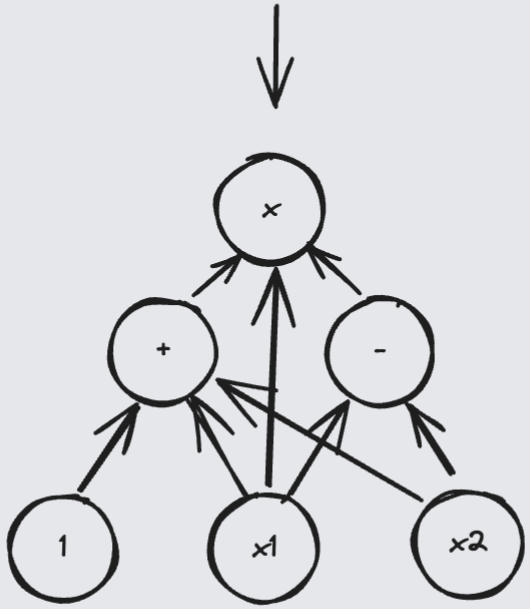
In this step we convert the original code, which may contain arbitrarily complex statements and expressions, into arithmetic circuit.

```
pragma circom 2.1.6;

template Example() {
  signal input x1;
  signal input x2;
  signal output out;

  out <== (1 + x1 + x2) * x1 * (x2 - x1);
}

component main = Example();
```



Rank one constrain system (R1CS)

- The witness vector is $1 \times n$ vector that contains the value of all the input variables, the output variable, and the intermediate values
- $out = x^2 y$
  - $v_1 = x \cdot x$
  - $out = v_1 \cdot y$
  - witness vector: $[1, out, x, y, v_1]$

Rank one constrain system (R1CS)

- An R1CS is a sequence of groups of three vectors (a, b, c) (each logic gate is converted into one group of vectors).
- The solution to an R1CS is a vector w, where w must satisfy the equation $(a \cdot w) \circ (b \cdot w) = c \cdot w$
- We can combine all equations into one equation $(A \cdot w) \circ (B \cdot w) = C \cdot w$
- For example $out = x^2 y$
  - $w = [1, out, x, y, v_1]$
  - $A = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$
  - $B = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix}$
  - $C = \begin{bmatrix} 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 \end{bmatrix}$

ZK R1CS

$$\begin{bmatrix} l_{1,1} & \dots & l_{1,m} \\ \vdots, & \ddots & \vdots \\ l_{n,1} & \dots & l_{n,m} \end{bmatrix} \begin{bmatrix} s_1 \\ \vdots \\ s_m \end{bmatrix} \odot \begin{bmatrix} r_{1,1} & \dots & r_{1,m} \\ \vdots, & \ddots & \vdots \\ r_{n,1} & \dots & r_{n,m} \end{bmatrix} \begin{bmatrix} s_1 \\ \vdots \\ s_m \end{bmatrix} = \begin{bmatrix} o_{1,1} & \dots & o_{1,m} \\ \vdots, & \ddots & \vdots \\ o_{n,1} & \dots & o_{n,m} \end{bmatrix} \begin{bmatrix} s_1 \\ \vdots \\ s_m \end{bmatrix}$$

$$\downarrow$$

$$\begin{bmatrix} l_{1,1} & \dots & l_{1,m} \\ \vdots, & \ddots & \vdots \\ l_{n,1} & \dots & l_{n,m} \end{bmatrix} \begin{bmatrix} [s_1 G_1]_1 \\ \vdots \\ [s_m G_1]_1 \end{bmatrix} = \begin{bmatrix} l_{1,1}[s_1 G_1] & +\dots & +l_{1,m}[s_m G_1] \\ \vdots, & \ddots & \vdots \\ l_{n,1}[s_1 G_1] & +\dots & +l_{n,m}[s_m G_1] \end{bmatrix} = \begin{bmatrix} \sum_{i=1}^m l_{1,i}[s_i G_1]_1 \\ \sum_{i=1}^m l_{2,i}[s_i G_1]_1 \\ \vdots \\ \sum_{i=1}^m l_{n,i}[s_i G_1]_1 \end{bmatrix}$$

$$\begin{bmatrix} r_{1,1} & \dots & r_{1,m} \\ \vdots, & \ddots & \vdots \\ r_{n,1} & \dots & r_{n,m} \end{bmatrix} \begin{bmatrix} [s_1 G_2]_2 \\ \vdots \\ [s_m G_2]_2 \end{bmatrix} = \begin{bmatrix} r_{1,1}[s_1 G_2]_2 & +\dots & +r_{1,m}[s_m G_2]_2 \\ \vdots, & \ddots & \vdots \\ r_{n,1}[s_1 G_2]_2 & +\dots & +r_{n,m}[s_m G_2]_2 \end{bmatrix} = \begin{bmatrix} \sum_{i=1}^m r_{1,i}[s_i G_2]_2 \\ \sum_{i=1}^m r_{2,i}[s_i G_2]_2 \\ \vdots \\ \sum_{i=1}^m r_{n,i}[s_i G_2]_2 \end{bmatrix}$$

ZK R1CS

$$\begin{bmatrix} l_{1,1} & \dots & l_{1,m} \\ \vdots & \ddots & \vdots \\ l_{n,1} & \dots & l_{n,m} \end{bmatrix} \begin{bmatrix} s_1 \\ \vdots \\ s_m \end{bmatrix} \odot \begin{bmatrix} r_{1,1} & \dots & r_{1,m} \\ \vdots & \ddots & \vdots \\ r_{n,1} & \dots & r_{n,m} \end{bmatrix} \begin{bmatrix} s_1 \\ \vdots \\ s_m \end{bmatrix} = \begin{bmatrix} o_{1,1} & \dots & o_{1,m} \\ \vdots & \ddots & \vdots \\ o_{n,1} & \dots & o_{n,m} \end{bmatrix} \begin{bmatrix} s_1 \\ \vdots \\ s_m \end{bmatrix}$$

$$\downarrow$$

$$\text{pairing}\left(\begin{bmatrix} \sum_{i=1}^m l_{i,1}[s_i G_1]_1 \\ \sum_{i=1}^m l_{i,1}[s_i G_1]_1 \\ \vdots \\ \sum_{i=1}^m l_{i,1}[s_i G_1]_1 \end{bmatrix}, \begin{bmatrix} \sum_{i=1}^m r_{i,1}[s_i G_2]_2 \\ \sum_{i=1}^m r_{i,1}[s_i G_2]_2 \\ \vdots \\ \sum_{i=1}^m r_{i,1}[s_i G_2]_2 \end{bmatrix} \right) \stackrel{?}{=} \text{pairing}\left(\begin{bmatrix} \sum_{i=1}^m o_{i,1}[s_i G_1]_1 \\ \sum_{i=1}^m o_{i,1}[s_i G_1]_1 \\ \vdots \\ \sum_{i=1}^m o_{i,1}[s_i G_1]_1 \end{bmatrix}, \begin{bmatrix} G_2 \\ G_2 \\ \vdots \\ G_2 \end{bmatrix} \right)$$

$$= \begin{bmatrix} [\text{pairing}(\sum_{i=1}^m l_{i,1}[s_i G_1]_1, \sum_{i=1}^m r_{i,1}[s_i G_2]_2)]_{12} \\ [\text{pairing}(\sum_{i=1}^m l_{i,2}[s_i G_1]_1, \sum_{i=1}^m r_{i,2}[s_i G_2]_2)]_{12} \\ \vdots \\ [\text{pairing}(\sum_{i=1}^m l_{i,n}[s_i G_1]_1, \sum_{i=1}^m r_{i,n}[s_i G_2]_2)]_{12} \end{bmatrix} \stackrel{?}{=} \begin{bmatrix} [\text{pairing}(\sum_{i=1}^m o_{i,1}[s_i G_1]_1, G_1)]_{12} \\ [\text{pairing}(\sum_{i=1}^m o_{i,2}[s_i G_1]_1, G_2)]_{12} \\ \vdots \\ [\text{pairing}(\sum_{i=1}^m o_{i,n}[s_i G_1]_1, G_2)]_{12} \end{bmatrix}$$

Quadratic Arithmetic Programs (QAP)

- A QAP is a polynomial encoding of R1CS.
- It turns many constraints into one polynomial identity.

QAP polynomial construction

- R1CS equation: $(A \cdot w) \circ (B \cdot w) = C \cdot w$
- Build a polynomial for each column in A, B, C and one polynomial for w
- For each vector v, construct a polynomial passing through points (i, v_i)
- Apply Lagrange interpolation to build such polynomials.

QAP polynomial construction

- $A \to A(x) = [a_1(x), a_2(x), \ldots, a_m(x)]$
- $B \to B(x) = [b_1(x), b_2(x), \ldots, b_m(x)]$
- $C \to C(x) = [c_1(x), c_2(x), \ldots, c_m(x)]$
- $(A \cdot w) \circ (B \cdot w) = C \cdot w \Rightarrow (A(x) \cdot w) \cdot (B(x) \cdot w) = C(x) \cdot w \; \forall x = 1, \ldots, n$

QAP checks

- $(A(x) \cdot w) \cdot (B(x) \cdot w) = C(x) \cdot w \; \forall x = 1, \ldots, n$
- $(A(x) \cdot w) \cdot (B(x) \cdot w) - (C(x) \cdot w) = 0 \; \forall x = 1, \ldots, n$
- $[(A(x) \cdot w) \cdot (B(x) \cdot w) - (C(x) \cdot w)] \vdots T(x) = (x-1) \ldots (x-n)$
- $\exists H(x) : (A(x) \cdot w) \cdot (B(x) \cdot w) - (C(x) \cdot w) = T(x) \cdot H(x)$

  Prover can calculate $H(x) = \frac{(A(x) \cdot w) \cdot (B(x) \cdot w) - (C(x) \cdot w)}{T(x)}$

Schwartz–Zippel lemma

The Schwartz–Zippel lemma ensures that in QAPs, if there is not a polynomial identity $(A(x) \cdot w) \cdot (B(x) \cdot w) \neq (C(x) \cdot w) + H(x)T(x)$, it will likely be detected by evaluating at a random point τ.

Adding homomorphic hiding

- Trusted setup: choose some $x = \tau$ and calculate $\tau G, \tau^2 G, \ldots, \tau^n G$

$$\pi_A = E(A(\tau) \cdot w) = a_d E(\tau^d) + a_{d-1} E(\tau^{d-1}) + \cdots + a_1 E(\tau) + a_0 = a_d(\tau^d G_1) + a_{d-1}(\tau^{d-1} G_1) + \cdots + a_1(\tau G_1) + a_0 G_1$$

$$\pi_B = E(B(\tau) \cdot w) = b_d E(x^d) + b_{d-1} E(x^{d-1}) + \cdots + a_1 E(x) + b_0 = b_d(x^d G_2) + b_{d-1}(x^{d-1} G_2) + \cdots + b_1(x G_2) + b_0 G_2$$

$$\pi_C = E(C(\tau) \cdot w + H(\tau) \cdot T(\tau)) = c_d E(\tau^d) + c_{d-1} E(\tau^{d-1}) + \cdots + c_1 E(\tau) + c_0 = c_d(\tau^d G_1) + c_{d-1}(\tau^{d-1} G_1) + \cdots + c_1(\tau G_1) + c_0 G_1$$

- Verifier checks pairing $e(\pi_A, \pi_B) = e(\pi_C, G_2)$

Thats all for today :)